Algoritmo di Prim - APA

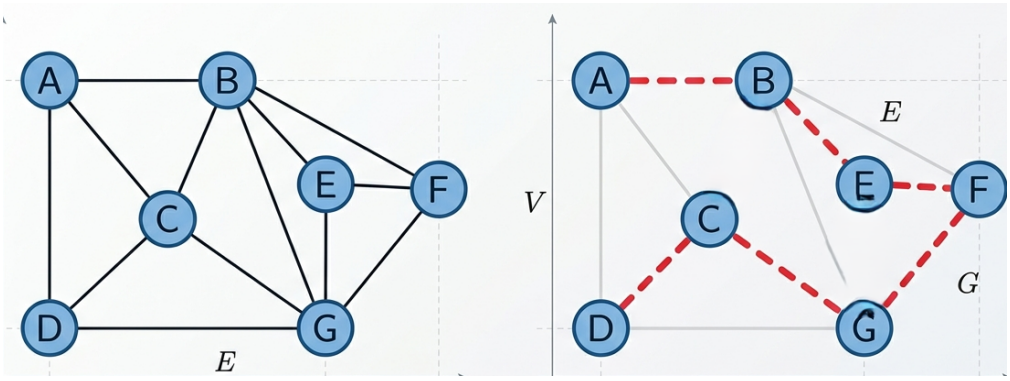
Fabio Nicolosi s7592267@studenti.unige.it

May 2026

1 Introduzione

prima di introdurre **l'algoritmo di Prim**, abbiamo bisogno di alcuni fondamenti teorici e dei loro significati:

1.1 Spanning Tree (Albero di Ricoprimento)

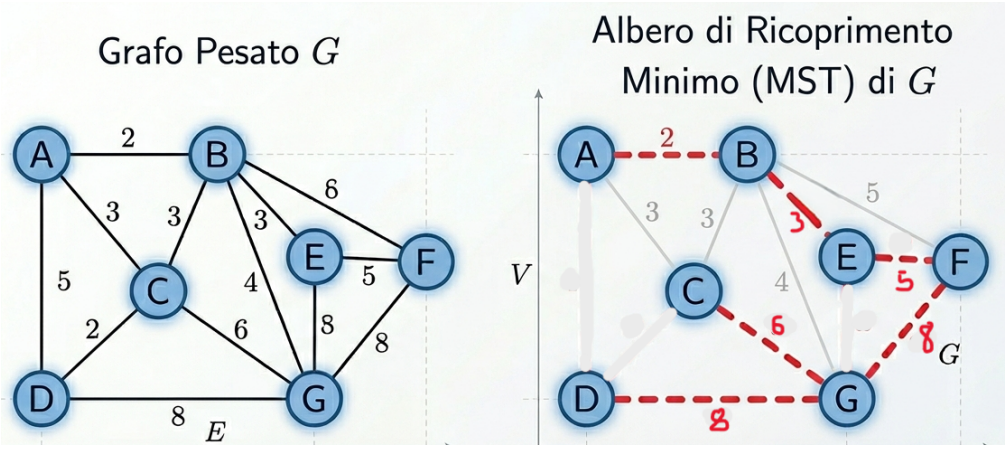


Un albero di ricoprimento, comunemente definito spanning tree, è un sottografo di un dato grafo connesso G che possiede tre proprietà fondamentali:

- Contiene tutti i nodi del grafo G
- Mantiene la connettività: garantisce che sia sempre possibile trovare un cammino per "viaggiare" da un qualsiasi nodo u a un qualsiasi altro nodo v.
- Assenza di cicli: è una struttura aciclica, il che significa che è impossibile partire da un nodo e ritornarvi senza dover percorrere a ritroso gli stessi archi.

Osservazione: Se il grafo G non è pesato, tutti gli alberi di ricoprimento possibili sono equivalenti dal punto di vista del costo. Questa equivalenza viene meno nei grafi pesati, dove ciascun albero può avere un costo totale differente.

1.2 Albero di Ricoprimento di Costo Minimo (MST)



Per comprendere questo concetto, introduciamo innanzitutto la definizione di **grafo pesato**: si tratta di un grafo in cui a ogni singolo arco viene associato un valore numerico detto peso o costo (che può rappresentare, a seconda dell'applicazione, una distanza, un tempo di percorrenza, un costo economico, ecc.).
In termini matematici, un grafo pesato viene denotato come $G = (V, E)$, dove la corrispondenza tra gli archi e i loro pesi è regolata da una funzione di costo $c : E \to \mathbb{R}$.

Dato un grafo G connesso, non orientato e pesato, l'Albero di Ricoprimento Minimo (o Minimum Spanning Tree - MST) è un sottoinsieme di archi che racchiude in sé le seguenti proprietà:

- **Essere un albero libero**: la struttura deve risultare completamente connessa e priva di cicli (aciclica).
- **Includere la totalità dei nodi**: l'albero deve contenere tutti i vertici appartenenti al grafo originale G.
- **Minimizzare il costo complessivo**: la somma dei pesi di tutti gli archi selezionati per formare l'albero deve essere la minima possibile rispetto a qualsiasi altra combinazione valida

---

2 Algoritmo

2.1 Scopo

L'algoritmo di Prim viene impiegato quando si ha la necessità di determinare in modo efficiente l'Albero di Ricoprimento Minimo (MST) di un dato grafo connesso e pesato. Questo algoritmo si basa su una strategia di tipo *greedy*: esso costruisce l'albero incrementale, un pezzo alla volta, e ogni volta che viene effettuata una scelta (aggiungendo un nodo e un arco alla soluzione), quest'ultima è da considerarsi definitiva e irrevocabile.

2.2 Funzionamento

```python
def algoritmo_prim(N, G, s=0):
    visit= [False]*N
    dist = [INF] * N

    parents= [NULL] *N

    parents[s]= NULL
    dist[s]=0
    Q=heapdict()

    for u in G:
        Q[u] = dist[u]


    while Q:
       u, Pu = Q.popitem()
       visit[u] = True
       for v, costo in G[u]:
           if not visit [v] and costo < dist[v]:
               parents[v]=u
               dist[v]=costo
               Q[v] = dist[v]

    return parents
```

Algoritmo implementato in Python

Come possiamo notare dallo pseudocodice, alla funzione vengono passati tre parametri: il numero totale di nodi N, il grafo di partenza G (rappresentato tramite una lista di adiacenza nella forma `G[u] = (v, costo))` e il nodo sorgente s, che indica l'indice del primo elemento da cui far partire la visita.
All'inizio del processo viene eseguita la fase di inizializzazione dei vettori di supporto:

- `visit`: un vettore booleano che tiene traccia dei nodi già inclusi nell'albero (neri). Viene inizializzato interamente a `False` (nodi non ancora visitati).
- `dist`: un vettore che memorizza la distanza minima di ciascun nodo dall'albero in costruzione. Tutti i suoi valori vengono inizialmente impostati a infinito (∞).
- `parents`: il vettore che conterrà la struttura dell'albero finale (le relazioni padre-figlio). Viene inizializzato inserendo il valore `NULL` per ogni posizione.

Subito dopo, definiamo il punto di partenza dell'algoritmo impostando il "genitore" di s come `NULL` e la sua distanza a 0, dato che la radice non ha nodi precedenti. Successivamente, viene creata la coda di priorità `Q`, nella quale vengono inseriti tutti i nodi del grafo con le rispettive distanze iniziali (che saranno tutte pari a ∞, eccetto per il nodo sorgente s che ha distanza 0).

2.2.1 Ciclo Principale (`while`)

Il ciclo `while` continua la sua esecuzione finché la coda `Q` contiene elementi.
A ogni iterazione, tramite la funzione `Q.popitem()`, viene estratto il nodo `u` che presenta la distanza minore in quel momento (al primo ciclo sarà proprio il nodo sorgente s). Appena estratto, il nodo viene marcato come visitato impostando `visit[u] = True`

2.2.2 Esplorazione dei Vicini (`for`)

L'algoritmo esamina quindi tutti gli archi adiacenti al nodo appena estratto `u`.
Per ogni nodo di arrivo `v` e per il rispettivo costo dell'arco che li unisce, viene effettuato un controllo: se il nodo `v` non è ancora stato visitato E il costo dell'arco corrente è inferiore alla distanza precedentemente memorizzata in `dist[v]`, allora l'algoritmo esegue le seguenti operazioni:

1. Aggiorna il padre di `v`, stabilendo che il suo predecessore ottimale è `u`
2. Aggiorna il valore della distanza minima memorizzata per quel nodo con il nuovo costo dell'arco
3. Modifica la priorità del nodo `v` all'interno della coda `Q`, aggiornandola con il nuovo valore di distanza calcolato

Al termine del ciclo `while`, la funzione restituisce il vettore `parents`, che descrive interamente l'Albero di Ricoprimento Minimo ottenuto.

2.3 Costo

Se indichiamo con n il numero di nodi e con m il numero di archi presenti nel grafo G, la complessità computazionale complessiva dell'algoritmo di Prim è pari a:

$$O((n+m)\log n)$$

2.3.1 Analisi Dettagliata dei Costi

Il costo totale dell'algoritmo è determinato principalmente dalle operazioni di accesso e manipolazione all'interno della coda di priorità (implementata tramite un min-heap). Ricordiamo che ogni singola operazione su un elemento all'interno di uno heap richiede un tempo logaritmico pari a $O(\log n)$.

A partire da questa osservazione, possiamo scomporre i costi delle diverse fasi dell'algoritmo:

- Inizializzazione della coda: $O(n \log n)$
- Il ciclo `while`: Il ciclo principale viene eseguito esattamente n volte, poiché ogni nodo viene inserito in coda ed estratto una sola volta. il costo totale di tutte le estrazioni nell'intero algoritmo è pari a $O(n \log n)$.
- Il ciclo `for`: All'interno del ciclo while, l'algoritmo esamina le adiacenze del nodo estratto. Nel corso di tutta l'esecuzione, ogni arco viene controllato al massimo per i suoi nodi adiacenti, traducendosi in m controlli complessivi. il costo totale massimo per la gestione degli archi è pari a $O(m \log n)$

Sommando i contributi di ciascuna fase ($O(n \log n)$ per le estrazioni e $O(m \log n)$ per gli aggiornamenti, otteniamo per l'appunto la complessità asintotica totale di $O((n+m)\log n)$

3 Output

3.1 Grafo-10-30

```
10
2 1 1.26
7 2 2.39
0 3 2.17
3 4 3.11
4 5 2.28
5 6 3.36
6 7 1.14
7 8 2.05
8 9 3.22
```

3.2 Grafo-30-150

```
30
0 1 1.97
0 2 1.45
0 3 2.95
1 4 1.22
1 5 2.61
2 6 2.1
2 7 1.17
3 8 2.52
3 9 1.11
4 10 2.3
4 11 1.21
5 12 1.27
5 13 2.27
6 14 3.48
6 15 1.37
7 16 1.67
7 17 2.88
8 18 3.84
8 19 2.73
9 20 2.19
9 21 3.93
10 22 1.14
11 23 3.58
13 24 1.87
14 25 1.43
16 26 1.35
18 27 1.93
19 28 3.45
21 29 1.54
```

```
INF= float('inf')
NULL = None

def leggi_grafo(nome_file):
    """
    Legge il grafo da un file ASCII.
    Formato:
    - Prima riga: N (numero di nodi)
    - Righe successive: u v costo
    """
    with open(nome_file, 'r') as f:
        linee = f.read().splitlines()

    if not linee:
        return 0, {}

    # N: numero di nodi (indicizzati implicitamente da 0 a N-1)
    N = int(linee[0].strip())

    # Inizializzazione della struttura dati a liste di adiacenza
    # G[u] conterr  una lista di tuple (vicino, costo)
    G = {i: [] for i in range(N)}

    # Lettura della sequenza di archi con costo
    for linea in linee[1:]:
        linea = linea.strip()
        if not linea:
            continue
        parti = linea.split()
        u = int(parti[0])
        v = int(parti[1])
        costo = float(parti[2])

        # Sendo il grafo non orientato, l'arco va inserito in entrambe le liste
        G[u].append((v, costo))
        G[v].append((u, costo))

    return N, G

def scrivi_output(nome_file, N, parent, G):
    """
    Scrive l'albero di ricoprimento minimo su file nello stesso formato di input.
    """
    with open(nome_file, 'w') as f:
        # Scrive il numero di nodi
        f.write(f"{N}\n")

        # Ricostruisce gli archi dell'MST usando il vettore dei parent
        for v in range(N):
            u = parent[v]
            if u is not None:
                # Cerca il costo dell'arco (u, v) nelle liste di adiacenza
                costo = next(c for vicino, c in G[u] if vicino == v)
                f.write(f"{u} {v} {costo}\n")

def algoritmo_prim(N, G, s=0):
    visit= [False]*N
    dist = [INF] * N

    parents= [NULL] *N

    parents[s]= NULL
    dist[s]=0
    Q=heapdict()

    for u in G:
        Q[u] = dist[u]


    while Q:
       u, Pu = Q.popitem()
       visit[u] = True
       for v, costo in G[u]:
            if not visit [v] and costo < dist[v]:
                parents[v]=u
                dist[v]=costo
                Q[v] = dist[v]

    return parents
```